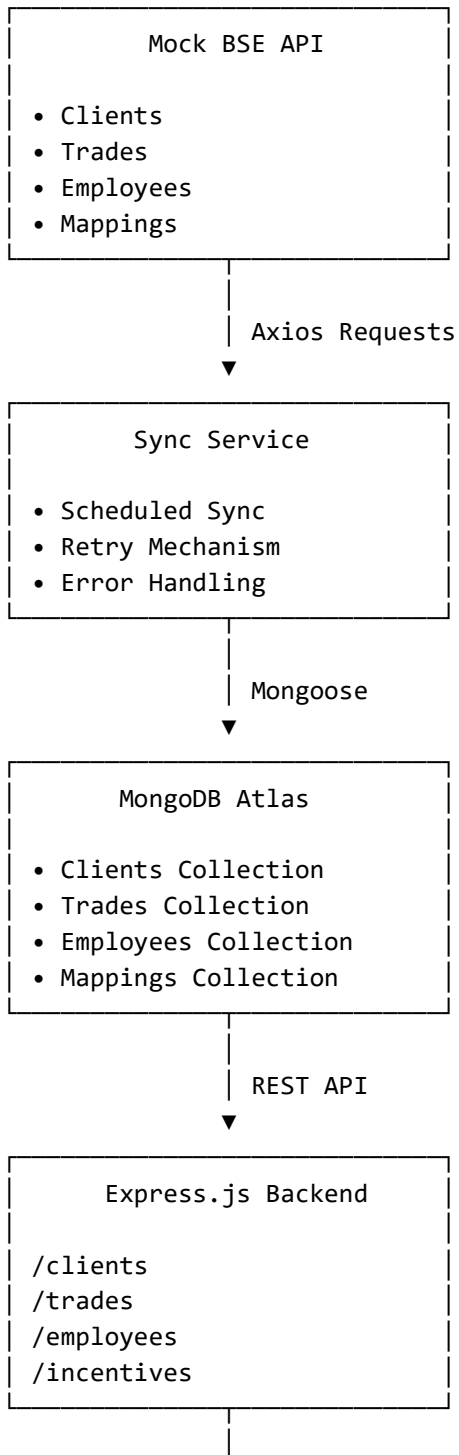
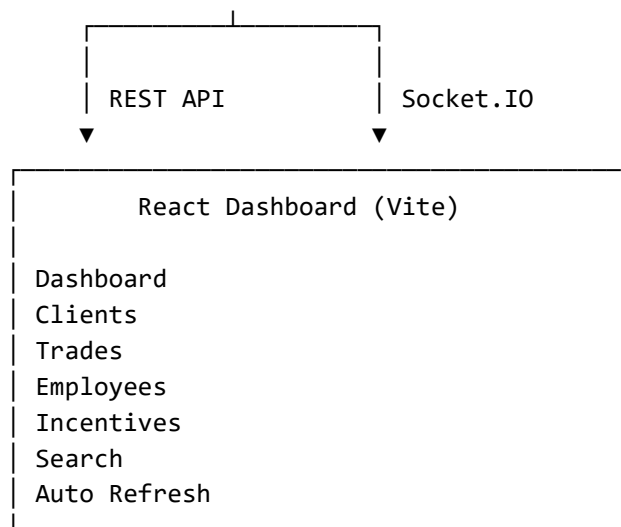


Mock BSE Dashboard – System Architecture

Architecture Diagram:





Architecture Reasoning

The application is divided into two independent parts: a **Mock BSE API** and an **Internal Dashboard**.

The Sync Service periodically fetches data from the Mock BSE API. A retry mechanism handles temporary API failures, ensuring reliable synchronization. The fetched data is stored in MongoDB Atlas so that the dashboard can continue serving the latest available data even if the Mock BSE API becomes unavailable.

Express.js exposes REST APIs that are consumed by the React frontend. Incentive calculations are performed on the backend using trade and employee mapping data.

Socket.IO is used to notify all connected dashboard clients whenever a synchronization cycle completes. Upon receiving the event, the frontend automatically reloads the latest data without requiring the user to refresh the browser.

This architecture separates data ingestion, storage, business logic, and presentation, making the application scalable, maintainable, and fault tolerant.

Running at 100× Data Volume

To support 100× more records:

- Add indexes on frequently queried fields such as `clientId`, `employeeId`, and `tradeId`.
- Replace full collection refresh with incremental synchronization.
- Implement pagination and server-side filtering.

- Use MongoDB aggregation pipelines for incentive calculations.
 - Deploy multiple backend instances behind a load balancer.
 - Introduce Redis caching for frequently accessed APIs.
 - Use background job queues for synchronization instead of a single process.
 - Enable horizontal scaling for the frontend and backend.
-